

# EE272 Homework 1

## Accelerator Design Space Exploration

Due: Jan 15, 2026, 11:59 pm

Released: Jan 8, 2026, 10:30 am

### 1 Overview

For this assignment, we will use and extend an optimizer to perform performance and power optimizations under an area constraint on an accelerator for convolutional neural networks (CNNs).

The CNN we will use for this assignment is ResNet-18 [1]. ResNet is composed of multiple residual building blocks that perform residual learning. This kind of residual network is easier to optimize, and can gain accuracy from increasing depth. A residual building block is shown in Figure 1, and is formally defined as:

$$y = F(x, W_i) + x \quad (1)$$

Here  $x$  and  $y$  are the input and output feature maps. The function  $F(x, W_i)$  is the residual mapping to learn. In the example shown in Figure 1,  $F(x, W_i) = W_2\sigma(W_1x)$ , where  $\sigma$  denotes the ReLU function, omitting the bias term. At the end,  $F(x) + x$  is performed with an identity shortcut connection and element-wise addition.

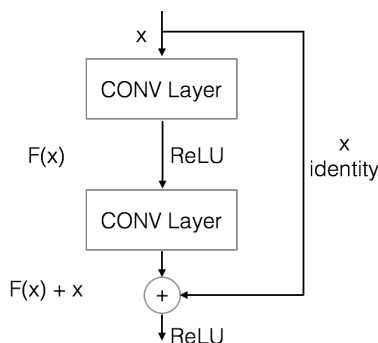


Figure 1: Residual building block [1].

By stacking multiple residual building blocks, ResNet-18 [1] can be constructed as shown in Figure 5. The identity shortcuts (equation 1) can be directly used when the input and output have the same number of feature maps. Note that, in the original paper, when the number of feature maps increase, two options are considered, but here we will just consider

option (A) in [1], which performs identity mapping, with extra zero entries padded for increasing dimensions.

**Required Reading:** Deep Residual Learning for Image Recognition [1]. Interstellar: Using Halide’s Scheduling Language to Analyze DNN Accelerators [2].

## 2 Optimizer

We provide an optimizer that can analyze and optimize the performance and power of the accelerator for each given convolutional (CONV) layer. The optimizer can be found at this [code repo](https://code.stanford.edu/ee272/ee272-interstellar) (<https://code.stanford.edu/ee272/ee272-interstellar>).

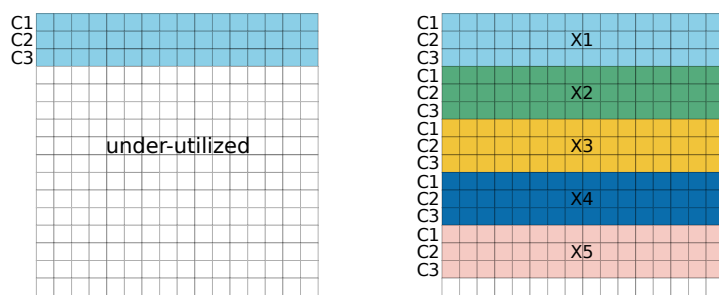
### 2.1 Terminology

Please read [2] to gain a better understanding of the optimizer and the terminology before using or modifying it. The terminology we will mostly use from the paper is summarized below:

**Blocking/tiling:** Assuming a multi-level memory hierarchy (e.g., register files, on-chip SRAM, and off-chip DRAM), we want to schedule the computation to maximize the data reuse in the near, smaller memories to lower overall energy cost. However to reduce the energy cost of the local memory, they need to remain small. Since all the data fetched — inputs, outputs, and weights — can be potentially reused, an optimal schedule must choose the best data reuse opportunities. The techniques of loop tiling and reordering, which together we refer to as loop blocking [3], transform the nested loops in CNN algorithm (Algorithm 1 in [2]) and generate different data access/reuse patterns for each memory level. We use loop tiling and reordering to block the computation, so that more data reuse can be captured at a lower level of the memory hierarchy (faster memory), that is, more data fetches can be served from the cheaper memory to reduce the references to the more expensive memories and improve the overall energy efficiency.

**Dataflow/spatial unrolling:** NN accelerators often exploit parallelism to improve performance, by using multiple processing elements (PEs) simultaneously. Essentially, they execute one or more loops in CNN algorithm (Algorithm 1 in [2]) in parallel through *spatial loop unrolling* or *dataflow*. The data access and communication patterns across the multiple PEs are determined by the spatially unrolled loops. Typically, the dataflow is carefully orchestrated so that data accesses to more expensive memories, including the storage in other PEs and large shared buffers, can be minimized.

**Replication:** Some dataflows, which unroll a single loop along each spatial dimension, can potentially result in under-utilizing computation resources. For instance, as illustrated in Figure 2(a), when unrolling a loop  $C$  with a size of 3 on the vertical dimension of a  $16 \times 16$  PE array, only 3 of the 16 rows of PEs are utilized, leaving the remaining PEs idle. To overcome this issue, in addition to unrolling loop  $C$ , another loop such as  $X$  is also unrolled by a factor of 5, as shown in Figure 2(b), improving the utilization ratio from  $3/16$  to  $15/16$ . Therefore, it is of great importance to support unrolling multiple loops onto one spatial dimension. This improvement is called *replication*.



(a) No replication.

(b) With replication.

Figure 2: Computation resource utilization can be improved by replication.

## 2.2 Getting Started

To start, make sure you follow the instructions in `HW0_farmshare_setup.pdf`. Here we assume you use the Singularity container approach.

Your default shell is `tcsh`. To verify:

```
$ ps -p $$
```

The interstellar repo has been updated to use `python3`. You can verify your Python version by:

```
$ python3 --version
```

You can download the code using the command below:

```
$ git clone https://code.stanford.edu/ee272/ee272-interstellar.git
```

To run the code, first, build the Python package by running the following from Interstellar repository:

```
$ python3 setup.py build
```

Set up this environment variable so that Python is able to find the scripts (substitute the actual path to the interstellar directory in the command below):

```
$ setenv PYTHONPATH "<path to your interstellar repo>/src"
```

You probably want to include this command in your `.tcshrc` file so that it is automatically set up for you in every shell.

The example commands in the following sections assume you are in the top level of the code repo directory. Navigate there by running:

```
$ cd ee272-interstellar
```

## 2.3 What Can It Do

This optimizer is designed to support the following tasks:

- **Blocking Search**

Given the memory system, number of computation resources (PEs) and the desired dataflow, search over the design space and report the best blocking, and the resulting power and performance. For example, to run this optimization for AlexNet CONV3 layer with batch size 16 using dataflow  $C | K$  (input channels and output channels unrolled), run the following command:

```
$ python3 ./tools/run_optimizer.py basic \
./examples/arch/3_level_mem_basic_example.json \
./examples/layer/alex_conv3_batch16.json \
-s ./examples/schedule/dataflow_C_K.json -v
```

- **Memory Search**

Given the number of computation resources (PEs) and the desired dataflow, search over the design space and report the best memory system configuration, the corresponding best blocking, and the resulting power and performance. The tool prints the output in the iteration order, and the user needs to match the optimal configuration based on the optimal energy value. To to run this optimization for AlexNet CONV3 layer with batch size 16 using dataflow  $C | K$ , run the following command:

```
$ python3 ./tools/run_optimizer.py mem_explore \
./examples/arch/3_level_mem_explore_example.json \
./examples/layer/alex_conv3_batch16.json \
-s ./examples/schedule/dataflow_C_K.json -v
```

- **Dataflow Search**

Given the memory system and number of computation resources (PEs), search over the design space and determine the impact of dataflow on power and performance. To run this for AlexNet CONV3 layer with batch size 16, run the following command:

```
$ python3 ./tools/run_optimizer.py dataflow_explore \
./examples/arch/3_level_mem_basic_example.json \
./examples/layer/alex_conv3_batch16.json \
-n user_defined_pickle_filename -v
```

You may wonder what those JSON files are? And where can you specify the configuration of the memory system and computation resources? Please read Section 2.4 for more details.

The above commands don't print any intermediate results. If you would like to dump intermediate results, you can use the **verbose** mode. You may also want to redirect the output to a log file for later review. For example:

```
$ python3 -u ./tools/run_optimizer.py basic
./examples/arch/3_level_mem_basic_example.json
./examples/layer/alex_conv3_batch16.json
-s ./examples/schedule/dataflow_C_K.json -v >& log
```

You can even use higher verbose mode to get more intermediate prints, for example **-vv** or **-vvv**.

## 2.4 Configuration File

To specify the configuration of the architecture, layer, or desired dataflow, you can create your own configuration files or modify the existing files under the **examples** folder. In this folder, you can find some examples of architecture configuration files under **arch** folder, layer configuration files under **layer** folder, and schedules specified in files under **schedule** folder. Each folder has a **README** file that specifies the required format for each type of configuration file.

### 2.4.1 Architecture Configuration

You can specify the number of memory levels, the size of each memory level, the energy cost per access at each level, and amount of parallelism at each level, the communication cost and the utilization threshold, etc. For example, for **3\_level\_mem\_basic\_example.json**, reproduced below the architecture will be configured as shown in Figure 3 with the memory reference cost marked on each interface.

```
{
  "mem_levels": 3,
  "capacity": [512, 131072, 1073741824],
  "access_cost": [0.96, 20, 200],
  "parallel_count": [256, 1, 1],
  "parallel_cost": [0.035],
  "precision": 16
}
```

- The **mem\_levels** field denotes the number of levels in the memory hierarchy, in this case 3 - register file, buffer and DRAM.

- The size of each memory level is configured using the **capacity** field. If **capacity** is set to [512, 131072, 1073741824], the size of the register file inside **each PE** is 512 B, the global buffer size is 128 KB, and the last one is the DRAM size.
- **access\_cost** is the energy cost for accessing a 16b word at different levels of memory. Here, 0.96, 20 and 200 are access energies in pJ for register file, buffer and DRAM respectively.
- The systolic array size can be configured using the **parallel\_count** field. **parallel\_count** of [256, 1, 1] means that there are 256 instances of the register file, and 1 instance of the buffer and DRAM, for example, as in the case of a  $16 \times 16$  systolic array as shown in Figure 3.

Figure 4 presents the different architectures generated by setting **parallel\_count** differently. If **parallel\_count** is set to [4, 1, 1], the parallelism occur at the register file level, thus four PEs with one level of register file will be generated, as shown in Figure 4(a). If **parallel\_count** is set to [1, 4, 1, 1], the parallelism occurs at the secondary memory level, thus four PEs with two levels of memory inside each PE will be generated, as shown in Figure 4(b). (The number of level one register files are also 4 in this case, cumulative product of higher level parallelism.) These two modes are the ones you will most likely use for this homework, in other words, your **parallel\_count** should be either  $[x, 1, 1]$  or  $[1, x, 1, 1]$  ( $x > 1$ ).

- **parallel\_cost** is the communication energy cost (in pJ) among adjacent parallel units (PEs).
- **precision** is the precision of the input feature maps, output feature maps and weights. The current version of the optimizer assumes the same precision for all these data.

There are two more options not specified in this example:

- **utilization\_threshold** specifies that the optimizer should only consider the designs that achieve a utilization ratio larger than this threshold. Its default value is 0.75.
- **replication**: **true/false** specifies whether to allow replication, default is true, .

For the case of **3\_level\_mem\_explore.json**, we will have a few additional parameters that the user can control on top of the previously mentioned configuration parameters.

- **explore\_points** Number points to iterate on from the starting configuration in powers of 2. For example, the first element, 5, means that the capacity of the register file is iterated till 512, starting from 32.
- **access\_cost\_scale** Indicates how much the access cost is scaled from iteration to iteration. For example, the access cost of a 64B register file would be the base access cost 0.05 times 2.

```

{
  "mem_levels": 3,
  "capacity": [32, 32768, 1073741824],
  "access_cost": [0.05, 3.84, 200],
  "static_cost": [0, 0, 0],
  "parallel_count": [256, 1, 1],
  "mac_capacity": 0,
  "parallel_mode": [1, 0, 0],
  "parallel_cost": [2],
  "capacity_scale": [2, 2],
  "access_cost_scale": [2, 1.25],
  "explore_points": [5, 4],
  "precision": 16
}

```

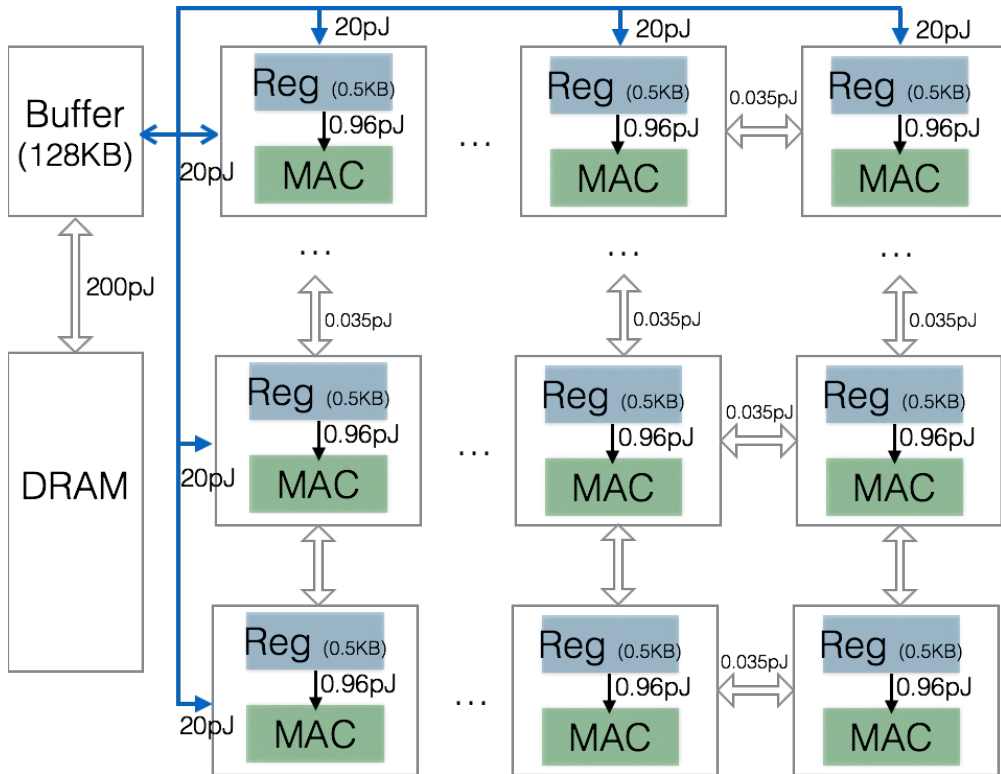
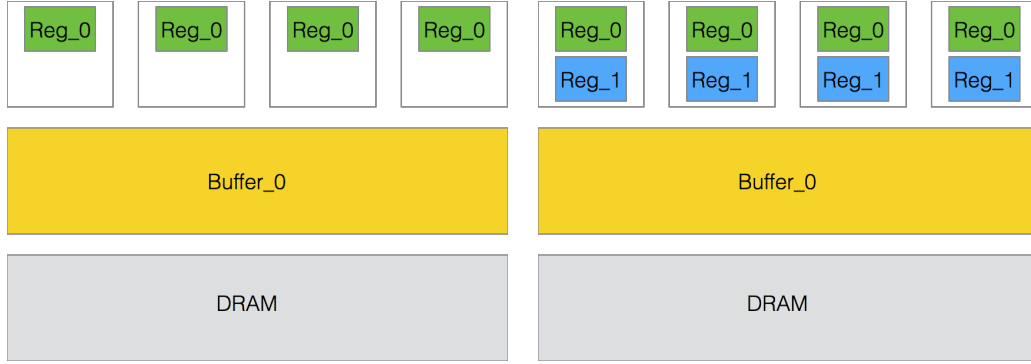


Figure 3: Systolic array.

#### 2.4.2 Layer Configuration

You can specify the feature map width and height, the number of input and output feature map channels, filter window width and height, batch size and stride. The feature map width



(a) Four PEs with one level of register file. (b) Four PEs with two levels of register file.

Figure 4: DNN accelerator architecture generated with setting `parallel_count` differently.

and height correspond to ones of output feature map, in other words, if you don't pad the input feature map, or have stride size more than 1, the output feature map will be smaller than input feature map. If you don't specify the stride in this file, it will assume a stride of 1.

```
{
  "fmap_width":13,
  "fmap_height":13,
  "input_fmap_channel":256,
  "output_fmap_channel":384,
  "window_width":3,
  "window_height":3,
  "batch_size":16,
  "stride_width":2,
  "stride_height":2
}
```

### 2.4.3 Schedule

You can impose some scheduling constraints at each memory level in this file. This is useful because the entire schedule space is usually so large that it will take a very long time to search over if you don't constrain it. For example, given the results from the dataflow paper, you can impose to use dataflow  $C \mid K$  as is done in the example `dataflow_C.K.json` file reproduced below.



```

{
  "schedule_hint": {
    "IC": {
      "level0":{
        "order":3,
        "partitioning_size":16
      }
    },
    "OC": {
      "level0":{
        "order":4,
        "partitioning_size":16
      }
    }
  }
}

```

In this case, the optimizer will only consider this particular dataflow for all the design choices. `partitioning_size` denotes how much the loop is unrolled. In this case, the input channels (IC or C) are unrolled or parallelized by 16 along one dimension of the PE array and output channels (OC or K) are unrolled along the other dimension of the PE array. The unroll factor (`partitioning_size`) of each dimension should not exceed the size of that dimension. For instance, if the ifmap only has 3 channels, you should not unroll the ifmap channel loop by 16. The loop orders of IC and OC are 3rd and 4th in this example, which means IC the 3rd innermost loop, and OC is the 4th innermost loop.

You may find that sometimes the optimizer can't find any valid design points, in this case, please check whether the specified dataflow and the utilization threshold match with each other. For example, if the unroll factor (`partitioning_size`) in the schedule file is too small to satisfy the utilization threshold, then no valid design points exist. To improve the utilization ratio of the PE array, you can turn on the replication mode, by adding loops to `partition_loops`, as shown in the example file `dataflow_C_K_replication.json` below.

```

{
  "schedule_hint": {
    "IC": {
      "level0": {
        "order": 3,
        "partitioning_size": 3
      }
    },
    "OC": {
      "level0": {
        "order": 4,
        "partitioning_size": 16
      }
    }
  },
  "partition_loops": ["OX", "ON"]
}

```

**Note:** Please be careful when setting `utilization_threshold`, the default value of 0.75 will be used when you leave this field blank. In this case, if your specified dataflow doesn't use more than 75% (the required utilization threshold) of the PEs, the optimizer will throw the error "Specified schedule doesn't satisfy the utilization threshold, please check partitioning\_size". In general, you would like `utilization_threshold` to be high to achieve high throughput, i.e. more than 50%. This would also help you reduce the search space. But do make sure you have sufficient unroll factors (`partitioning_size`) to achieve such high utilization. One easy way to avoid this problem is by always turning on replication in schedule file when setting high utilization threshold, such as shown in `dataflow_C_K.replication.json`.

## 2.5 Interpreting Optimizer Outputs

Here is some example output from the optimizer for the blocking search example from Section 2.3:

```

python3 ./tools/run_optimizer.py basic
↪ ./examples/arch/3_level_mem_basic_example.json
↪ ./examples/layer/alex_conv3_batch16.json -s
↪ ./examples/schedule/dataflow_C_K.json -v

Optimal_Energy_(pJ): 13177744588.8
Runtime_(cycles): 9345024.0
16b_Access_Energy_[RegisterFile(s),Buffer,DRAM]_(pJ):
    ifmap: [0.96, 20, 200]
    ofmap: [0.96, 20, 200]
    filter: [0.96, 20, 200]
PE_Access_Cost_(pJ):
    ifmap: 0.035
    ofmap: 0
    filter: 0
Tiles_Accessed_from_[RegisterFile(s),Buffer,DRAM]_in_Layer:
    ifmap: [3456.0, 4.0, 4.0]
    ofmap: [4607, 31.0, 1.0]
    filter: [2704, 8.0, 8.0]
Tiles_Accessed_from_[RegisterFile(s),Buffer,DRAM]_PEs_in_Layer:
    ifmap: [64.0, 256.0, 8.0]
    ofmap: [0.0, 0.0, 0.0]
    filter: [0.0, 0.0, 0.0]
Memory_Bank_Size_List_When_Parallelized/Unrolled_
↪ [RegisterFile(s),Buffer,DRAM]_(bytes):
    ifmap: [45, 450.0, 57600.0]
    ofmap: [78, 2028.0, 64896.0]
    filter: [54, 54, 3456.0]
Memory_Block_Size_List_When_NOT_Parallelized/Unrolled_
↪ [RegisterFile(s),Buffer,DRAM]_(bytes):
    ifmap: [11520, 7200.0, 921600.0]
    ofmap: [19968, 32448.0, 1038336.0]
    filter: [13824, 13824, 884736.0]
Layer_Size_(number_of_pixels):
    ifmap: 921600
    ofmap: 1038336
    filter: 884736

```

```

Best_Schedule_(innermost_loop_to_outermost_loop):
    Level_Number: 0
        Loop_Name: FX, Loop_Bound: 3, Unrolling: 1
        Loop_Name: FY, Loop_Bound: 3, Unrolling: 1
        Loop_Name: OY, Loop_Bound: 13, Unrolling: 1
        Loop_Name: OC, Loop_Bound: 6, Unrolling: 16
        Loop_Name: IC, Loop_Bound: 1, Unrolling: 16
    Level_Number: 1
        Loop_Name: OX, Loop_Bound: 13.0, Unrolling: 1
        Loop_Name: ON, Loop_Bound: 2, Unrolling: 1
    Level_Number: 2
        Loop_Name: OC, Loop_Bound: 4.0, Unrolling: 1
        Loop_Name: IC, Loop_Bound: 16.0, Unrolling: 1
        Loop_Name: ON, Loop_Bound: 8.0, Unrolling: 1
Elapsed_Time_(s): 55.82

```

- **Optimal\_Energy\_(pJ)** is the optimal total energy of the layer. It is helpful to evaluate the design points.
- **Runtime\_(cycles)** is the runtime of the layer in clock cycles. It is helpful to evaluate the design points.
- **16b\_Access\_Energy\_[RegisterFile(s),Buffer,DRAM]\_(pJ)** is the energy cost for accesses at different levels of memory. The numbers 0.96, 20, 200 are same as that specified in the architecture configuration file to access register file, buffer and DRAM respectively.
- **PE\_Access\_Cost\_(pJ)** is the cost to access adjacent PEs. If there is no replication used, the access cost into an adjacent PE's register file is  $0.96 + 0.035$ . However, sometimes when replication is used, the data has to be fetched from non-adjacent PEs, which takes multiple hops. Going through  $n$  hops takes  $n*0.035$  (pJ), so in that case, the cost will not be zero.
- **Tiles\_Accessed\_from\_[RegisterFile(s),Buffer,DRAM]\_in\_Layer** is the number of ifmap, ofmap, and filter **tile** accesses for each level of memory.
- **Memory\_Bank\_Size\_List\_When\_Parallelized/Unrolled**  
**\_ [RegisterFile(s),Buffer,DRAM]\_(bytes)** is the size of the memory required at each level for ifmap, ofmap and filter blocks when parallelized/unrolled. In this example, the register file level is unrolled, the minimum register file size required inside each PE is  $(45 + 78 + 54)*\text{precision}$ . If you use  $16*16$  PEs, then the total register file size is  $16*16*(45 + 78 + 54)*\text{precision}$ .
- **Memory\_Block\_Size\_List\_When\_NOT\_Parallelized/Unrolled**  
**\_ [RegisterFile(s),Buffer,DRAM]\_(bytes)** is the size of the memory required at each

level for ifmap, ofmap and filter blocks when not parallelized/unrolled. In this example, the minimum memory size required at the buffer level is  $(7200 + 32448 + 13824) \times \text{precision}$ .

- `Layer_Size_(number_of_pixels)` is a list with size (in number of pixels) of ifmap, ofmap, and filter of the layer. These are obtained from the layer configuration file. For example,

```
input_fmap_size = input_fmap_width * input_fmap_height * input_fmap_channel
* batch_size = (13 + 3 - 1) * (13 + 3 - 1) * 256 * 16 = 921600
```

- `Best_Schedule_(innermost_loop_to_outermost_loop)` is the most optimal loop blocking (ordering + tiling) the optimizer generates given the memory configuration, computation resources, and dataflow. It represents the tiled loop nests for the convolution. The list goes from innermost loop to the outermost loop. Using the above output as an example, the loop nest after using this schedule is:

```
# --- DRAM level below ---
for on = 0:7
  for ic = 0:15
    for oc = 0:3
      # --- Buffer level below ---
      for on = 0:1
        for ox = 0:12
          # --- Register file below ---
          parallel_for ic = 0:15 # unroll ic by 16
          for ic = 0:0
            parallel_for oc = 0:15 # unroll oc by 16
            for oc = 0:5
              for oy = 0:12
                for fy = 0:2
                  for fx = 0:2
```

- To make it easier to parse the results, the tool also dumps the optimized run time and energy in basic search mode in a JSON file (`result_<current_time>.json`) for you to inspect or process. If you prefer another name, you can specify the result file name with the option `-j <RESULT_JSON_FILE_NAME>` when you invoke `interstellar`.

### 3 Problem

Before designing and manufacturing an accelerator, doing design space exploration of potential designs is essential. In this class, we are creating an accelerator specifically for ResNet-18, so we are using an optimization tool to try and make decisions about the most efficient architecture given the information we have about the application.

The problem for this assignment is to leverage the provided optimizer to find the best architecture configuration and the scheduling for each layer to optimize the overall energy and performance for ResNet-18 under certain constraints. The accelerator you are designing runs a single ResNet layer at a time, and the architecture of that accelerator cannot change for each layer. The provided optimizer can analyze the energy and performance for a single layer. To optimize the energy and performance of the entire network, you will need to run the optimizer on multiple layers of ResNet-18. Scripting this in Python/Shell or modifying the optimizer code itself are recommended if you want to run many layer optimizations at the same time. **Note:** The ResNet layers in the examples folder are not ResNet-18, you will need to create those yourself.

The `basic` and `mem_explore` features of the optimizer are the ones you will likely leverage. You will find the optimal energy (pJ) and runtime (cycles) in the output printout and the resulting JSON file. Those numbers will be the most helpful ones to evaluate the design points.

### 3.1 Area Constraint

You can typically scale performance by simply by creating a larger accelerator, therefore, we require you to operate within an area constraint. The total area of your accelerator can NOT exceed 2 mm<sup>2</sup>. Using large or more memory can capture more data reuse and improve the overall energy efficiency, but also requires a larger area. The energy cost models are presented in Table 1 and Table 2. To compute the total area, use the area cost model provided in equation 2. The register file and buffer sizes used are what you have specified for `capacity` inside your architecture configuration file.

$$\text{total\_area} = \text{sum}(\text{area}(\text{MAC})) + \text{sum}(2 \times \text{area}(\text{Register File})) + 2 \times \text{area}(\text{Buffer}) \quad (2)$$

Assume that with the peripherals of an SRAM, the effective size of an SRAM bit cell is 0.2  $\mu\text{m}^2$ ; this value can be used to calculate the size of the buffers. For a single bit register, the effective size is 2  $\mu\text{m}^2$ ; this value can be used to calculate the effect of registers in the PE on area. Assume the size of 16b multiply add unit is around 1000 single bit register cells. Please remember, if the register file or buffer is in each of the parallelized PEs, the total area for this level is the summation of the area of all PEs. Also remember, when calculating the number of register files, it is the cumulative product of high level parallelism. So this means that the design in Figure 4(b) has 1\*4\*1\*1=4 level-one register files and 4\*1\*1=4 level-two register files, for a total of 8 register files. **Note:** Since we are using double buffer scheme for all memory levels, the actual memory size should be double the desired memory size.

DRAM area need not to be included in the total area of 2 mm<sup>2</sup>. But the energy cost of DRAM access should be considered, accessing 16b from DRAM costs 200pJ.

### 3.2 Considerations

There are multiple considerations you will likely need to take into account; please write down what you have considered and experimented with in your report.

- All the layers should use the same architecture for this homework. Specifically, the computation resource and memory system have to be the same for all layers. But the scheduling, including blocking and dataflow, don't have to be the same.
- Assume the accelerator computes one layer at a time. In other words, you don't have to consider multi-layer scheduling, which maps multiple layers together on chip at the same time. If you want to do that, please modify the optimizer accordingly, and clearly describe what you considered and how you did this optimization.
- Jointly optimizing the entire network may require a long search cycle. Try to find some ways to speedup the search process. For example, you may find that some layers have the same configuration. Don't repeat the search process for the same layer configuration. And think about whether there are any layers that dominate the overall energy efficiency or performance? Since this problem is **non-convex**, and searching to find the optimal points can take a very long time. **But if you try to optimize those critical layers first, greedy search may give you a close-to-optimal result, which will be sufficient for this homework.**
- Assume the precision we use is 16 bit. Assume the systolic array is always square, which means the number of PEs should be square numbers.
- The input size of the first layer is  $224 * 224 * 3$  (image width \* image height \* image channel), from the original ResNet paper.
- Assume the input batch size is 16 images.
- Increasing the number of PEs can improve the performance, but there is a trade-off here. The design we will give doesn't initially support replication, so it might be interesting to see the performance with and without replication (you can turn on and off replication mode to check how it can improve the performance and energy efficiency).
- To speedup the search process, there are some useful approximations you can make. One is that you can use the conclusion from [2], which is that many dataflows can achieve similar energy efficiency, thus you can just choose one dataflow, i.e. dataflow  $C \mid K$ , and only optimize blocking and architecture configurations. The other assumption you can use to reduce the design space is to only search the memory size that is power of 2, thereby converting the large continuous search space into a small discrete space.
- What is the pattern in the optimal memory system configurations? For example, what is the range of the size ratio between each adjacent memory levels? Does adding another register file level improve the energy efficiency? Will this pattern hold for other networks?
- Other considerations. You are encouraged to write up any other considerations or experiment other optimizations. For example, multiple level parallelism, off-chip or on-chip buffer bandwidth analysis, layout transformation between adjacent layers, etc.

Table 1: Cost model for SRAM buffer (16b).

| Size           | 8KB | 16KB | 32KB | 64KB | 128KB | 256KB | 512KB |
|----------------|-----|------|------|------|-------|-------|-------|
| Energy( $pJ$ ) | 4   | 6    | 9    | 13.5 | 20.2  | 30.4  | 45.6  |

Table 2: Cost model for register file (16b).

| Size           | 16B  | 32B  | 64B  | 128B | 256B | 512B |
|----------------|------|------|------|------|------|------|
| Energy( $pJ$ ) | 0.03 | 0.06 | 0.12 | 0.24 | 0.48 | 0.96 |

### 3.3 Code Submission and Grading

Please submit the code as a ZIP file to Gradescope under assignment HW1-Code. This can be the scripts you ran, or your modification of the original optimization code. You must also submit a report in **PDF** format to Gradescope under assignment HW1-Report. **You should use the template provided on Canvas** to answer these questions:

1. Start with your basic approach. How did your group decide to attack the problem. How were you going to find hardware that works for the overall problem, when the tool only optimizes a single layer at a time? What part of the design space did you explore, and why did you choose that subspace? How did you decide on which layers to look at first, and why did you choose these layers?
2. Given your strategy for what you wanted to do, how did you implement that strategy? What scripts did you write/modify? How did you fit in the area constraint? How did you plan work (be honest). This section should explain how to ran your scripts.
3. Present your results. Here it is better if you present what your results mean. That is, rather than just presenting a table of numbers, try to talk about important trends you see from the results – are many designs very similar to each other, or is there a strong optimal point; what is the most critical aspect to optimize; does the area constraint matter; etc. Include interesting plots in the report.
4. Finally, summarize the following parameters of your optimal  $2\text{ mm}^2$  design in Table 3. **Energy and cycles in the table must correspond to that of the entire ResNet-18 network (all CONV layers) on your accelerator.** Separately report the breakdown of energy between different layers for partial credit.

The grading of this assignment will be based on the overall performance and energy efficiency you achieve, and the report you write. Having a thorough analysis and interesting insights will be important to get higher score.



Table 3: Optimal accelerator parameters. If you have more levels in the memory hierarchy specify each of their sizes and clearly state if the size is per PE or the total for all PEs.

|                             | Value for your design |
|-----------------------------|-----------------------|
| Number of PEs               |                       |
| Register File Size (Bytes)  |                       |
| Global Buffer Size (KBytes) |                       |
| Energy (mJ)                 |                       |
| Cycles (million)            |                       |
| Area ( $mm^2$ )             |                       |

## References

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [2] Xuan Yang, Mingyu Gao, Qiaoyi Liu, Jeff Setter, Jing Pu, Ankita Nayak, Steven Bell, Kaidi Cao, Heonjae Ha, Priyanka Raina, Christos Kozyrakis, and Mark Horowitz. Interstellar: Using halide’s scheduling language to analyze dnn accelerators. *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020.
- [3] Xuan Yang, Jing Pu, Blaine Burton Rister, Nikhil Bhagdikar, Stephen Richardson, Shahar Kvatinsky, Jonathan Ragan-Kelley, Ardavan Pedram, and Mark Horowitz. A systematic approach to blocking convolutional neural networks. *arXiv preprint arXiv:1606.04209*, 2016.

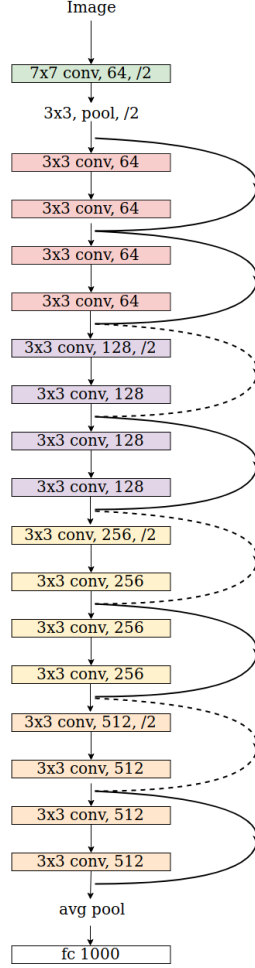


Figure 5: ResNet-18 [1].

## 4 Appendix

This section will describe exploring four levels of memory with Interstellar, namely two levels of register files, the buffer, and the DRAM. Interstellar is capable of making these explorations, but there are a few syntactical differences in the input configuration files and commands.

In addition, this problem is more constrained than the 3-level case and there is a greater chance that the code will finish without a mapping point found. Carefully consider the system constraints to ensure the search is worthwhile. One such constraint is that the Register File 0 capacity should be less than or equal to the Register File 1 capacity. This arrangement greatly increases the chance of finding a valid mapping point.

### 4.1 Input Configuration Files

To create a basic architecture file for the four-level case, the following changes are necessary.

- Change the number of memory levels to 4.

- Add the second register file to capacity, making the list [RegisterFile0, RegisterFile1, Buffer, DRAM].
- Add the access cost of the new register level to the list following the same ordering convention previously listed.
- Add parallelization directions for the new register file level to the list following the same ordering convention previously listed.

The following is an example of a basic architecture configuration file.

```
{
  "mem_levels": 4,
  "capacity": [16, 32, 131072, 1073741824],
  "access_cost": [0.03, 0.12, 20, 200],
  "parallel_count": [1, 256, 1, 1],
  "parallel_cost": [0.035],
  "precision": 16
}
```

The following items need to be added to the architecture file to use the memory explore function for the 4-level case.

- Add an explore\_points entry that instructs the optimizer to explore around the defined memory points for each level of memory.
- Add a capacity\_scale entry that informs the scaling for each explored point.
- Add an access\_cost\_scale entry to inform the optimizer how the memory access cost changes at the new sizing point.

An example architecture file is shown here with all entries necessary for the memory explore function.

```
{
  "mem_levels": 4,
  "capacity": [16, 32, 131072, 1073741824],
  "access_cost": [0.03, 0.12, 20, 200],
  "parallel_count": [1, 256, 1, 1],
  "parallel_cost": [0.035, 0.035, 0, 0],
  "precision": 16,
  "explore_points": [3, 4, 4],
  "capacity_scale": [2, 2, 2],
  "access_cost_scale": [2, 2, 2]
}
```

To create a schedule file for the four-level architecture, the schedule hint must specify that the IC and OC channel partitions should be in the level1 memory. If this is not specified, interstellar will not produce an optimal solution. An example schedule file is shown below.

```
{
  "schedule_hint": {
    "IC": {
      "level1": {
        "order": 3,
        "partitioning_size": 16
      }
    },
    "OC": {
      "level1": {
        "order": 4,
        "partitioning_size": 16
      }
    }
  }
}
```

The layer files do not require changes to use the four-level memory architecture.

## 4.2 Running the Optimizer

To run the four-level exploration, the following commands are used.

```
python3 ./tools/run_optimizer.py basic
./examples/arch/4_level_mem_basic.json
./examples/layer/alex_conv3_batch16.json -s
./examples/schedule/dataflow_C_K_4_level.json -v
```

Note the different setting for the memory explore command.

```
python3 ./tools/run_optimizer.py mem_explore_4
./examples/arch/4_level_mem.json
./examples/layer/alex_conv3_batch16.json -s
./examples/schedule/dataflow_C_K_4_level.json -v
```

The dataflow\_explore command is similar to the previous usage.

```
python3 ./tools/run_optimizer.py dataflow_explore  
./examples/arch/4_level_mem.json  
./examples/layer/alex_conv3_batch16.json -s  
./examples/schedule/dataflow_C_K_4_level.json -v
```